

平时作业

问题 1：浮点数有没有移位操作和扩展操作？为什么？

严格来说，浮点数没有整数意义上的直接移位操作，但存在浮点语义下的缩放操作，也存在扩展操作。

整数可以直接看成一个二进制位串。对整数左移一位，通常相当于乘以 2；右移一位，通常相当于除以 2。因此，整数移位可以直接在位级别完成。从硬件角度看，整数移位一般由 CPU 中的移位器完成，电路结构相对简单，执行效率较高。

浮点数的情况不同。常见浮点数采用 IEEE 754 标准表示，由符号位、阶码和尾数组成，其数值大致可以表示为：

$$(-1)^{\text{符号位}} * \text{尾数} * 2^{\text{阶码}}$$

如果直接把整个浮点数编码进行左移或右移，就会同时改变符号位、阶码和尾数的含义，得到的结果通常不是原数乘以 2 或除以 2，而是另一个完全不同的浮点编码，甚至可能成为非法或特殊编码。因此，浮点数不能像整数一样使用简单的整体位移来表示数值缩放。

不过，从数学意义上看，浮点数可以实现类似“移位”的效果。例如 $x * 2^n$ 可以看作浮点意义下的缩放。它的本质不是移动整个二进制编码，而是主要调整阶码，同时还要处理规格化、舍入、溢出和下溢等情况。在 C 语言中，`<<` 和 `>>` 运算符只适用于整数类型，不能用于浮点数。如果需要计算浮点数乘以 2 的 n 次方，可以使用标准库函数 `ldexp()` 或 `scalbn()`。

```
double y = ldexp(x, n);    // 计算 x * 2^n
```

浮点数也有扩展操作。例如从单精度 `float` 转换为双精度 `double`。这种扩展不是简单补 0 或补符号位，而是按照浮点格式重新编码：保持数值尽量不变，调整阶码偏置，增加尾数精度，并正确处理 `+0`、`-0`、无穷大和 NaN 等特殊值。

综上，浮点数没有普通整数那种直接的移位操作；如果要实现乘以 2 的幂，应使用浮点缩放操作。浮点数有扩展操作，但扩展过程需要遵循浮点格式，而不是简单地进行位扩展。

问题 2：为什么整数除 0 会发生异常而浮点数不会？

整数除 0 会发生异常，是因为整数系统中没有合适的值可以表示除 0 的结果。

例如：

```
1 / 0
```

这个表达式在数学上没有定义。在普通整数类型中，也不存在“正无穷大”“负无穷大”或“不是一个数”这样的结果。因此，CPU 执行整数除法时，如果发现除数为 0，通常会触发除零异常，由操作系统或运行时环境进行处

理。

浮点数除 0 的处理方式不同。IEEE 754 浮点标准定义了若干特殊值，可以表示无穷大和 NaN。因此，浮点除 0 通常可以得到一个符合标准的结果，而不是立即中断程序：

```
1.0 / 0.0    // +Infinity
-1.0 / 0.0   // -Infinity
0.0 / 0.0    // NaN
```

其中，正数除以 +0.0 可以得到 +Infinity，负数除以 +0.0 可以得到 -Infinity，而 0.0 / 0.0 得到 NaN，表示“不是一个数”。浮点数还区分 +0.0 和 -0.0，因此结果的符号也可能受到影响。

对应的实验程序如下：

```
#include "stdio.h"

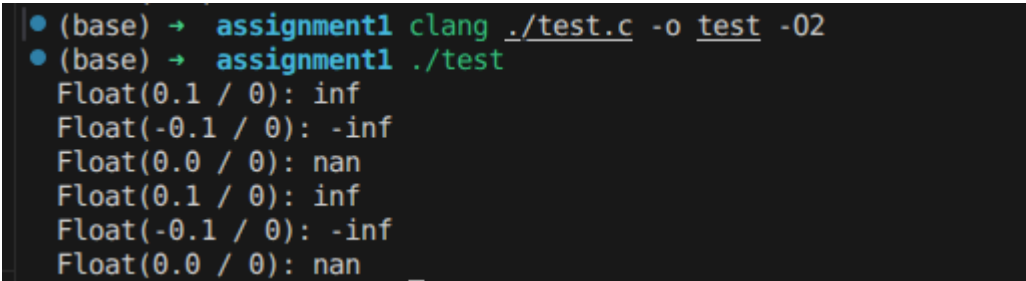
int main() {
    float a = 0.1;
    float b = -0.1;
    float c = 0.0;

    printf("Float(0.1 / 0): %f\n", a / 0);
    printf("Float(-0.1 / 0): %f\n", b / 0);
    printf("Float(0.0 / 0): %f\n", c / 0);

    printf("Float(0.1 / 0): %.20f\n", a / 0);
    printf("Float(-0.1 / 0): %.20f\n", b / 0);
    printf("Float(0.0 / 0): %.20f\n", c / 0);

    return 0;
}
```

实际运行结果如图 2 所示：



```
(base) → assignment1 clang ./test.c -o test -O2
(base) → assignment1 ./test
Float(0.1 / 0): inf
Float(-0.1 / 0): -inf
Float(0.0 / 0): nan
Float(0.1 / 0): inf
Float(-0.1 / 0): -inf
Float(0.0 / 0): nan
```

对应输出为：

```
Float(0.1 / 0): inf
Float(-0.1 / 0): -inf
Float(0.0 / 0): nan
Float(0.1 / 0): inf
```

```
Float(-0.1 / 0): -inf
Float(0.0 / 0): nan
```

由运行结果可知，正浮点数除以 0 得到 `inf`，负浮点数除以 0 得到 `-inf`，而 `0.0 / 0` 得到 `nan`。即使使用 `%.20f` 指定输出 20 位小数，特殊值仍然以 `inf`、`-inf` 和 `nan` 的形式输出。

这种设计具有工程意义。在科学计算、图形渲染和大规模数值计算中，如果某个中间步骤出现除 0，程序往往不希望立刻崩溃，而是希望先产生 `Infinity` 或 `NaN`，让计算流程继续进行，最后再由程序员检查结果或异常标志。

需要注意的是，浮点除 0 并不是完全没有异常。它在 IEEE 754 中仍属于浮点异常，通常会设置 `divide-by-zero` 等异常标志。也就是说，程序表面上没有崩溃，但 CPU 的浮点状态字或状态寄存器中已经记录了这次异常。只是默认情况下，浮点异常通常采用“不陷入”的处理方式，不会像整数除 0 那样立即进入异常处理流程。

因此，整数除 0 会发生异常，是因为整数没有对应的结果表示；浮点数除 0 通常不会让程序直接异常退出，是因为 IEEE 754 提供了无穷大、NaN 和异常标志机制。

问题 3：为什么同一个实数赋值给 float 型变量和 double 类型变量，输出结果会有所不同？

同一个实数赋值给 `float` 和 `double` 后输出不同，根本原因是二者的精度不同。

在常见的 IEEE 754 浮点格式中，`float` 通常占 32 位，有效精度约为 6 到 7 位十进制数字；`double` 通常占 64 位，有效精度约为 15 到 16 位十进制数字。因此，`double` 能保存更多有效位，通常比 `float` 更接近真实值。

很多十进制小数在二进制中无法被有限位数精确表示。例如 `0.1` 在二进制中是无限循环小数，所以无论赋值给 `float` 还是 `double`，实际保存的都只是近似值：

```
float a = 0.1;
double b = 0.1;
```

由于 `float` 和 `double` 的尾数位数不同，它们舍入后得到的二进制近似值也不同。以 `0.1` 为例：

类型	原始数值	实际存储的十六进制表示	十进制展开
<code>float</code>	<code>0.1</code>	<code>0x3DCCCCD</code>	<code>0.10000000149011611938</code>
<code>double</code>	<code>0.1</code>	<code>0x3FB999999999999A</code>	<code>0.10000000000000000555</code>

如果输出时只保留较少的小数位，二者可能看起来相同：

```
printf("%f\n", a);
printf("%f\n", b);
```

对应的实验程序如下：

```
#include "stdio.h"

int main() {
    float a = 0.1;
    double b = 0.1;

    printf("Float: %f\n", a);
    printf("Double: %f\n", b);

    printf("Float: %.20f\n", a);
    printf("Double: %.20lf\n", b);

    return 0;
}
```

实际运行结果如图 3 所示：

```
• (base) → assignment1 clang ./test.c -o test -O2
• (base) → assignment1 ./test
Float: 0.100000
Double: 0.100000
Float: 0.10000000149011611938
Double: 0.10000000000000000555
```

对应输出为：

```
Float: 0.100000
Double: 0.100000
Float: 0.10000000149011611938
Double: 0.10000000000000000555
```

由运行结果可知，当默认输出 6 位小数时，`float` 和 `double` 都显示为 `0.100000`，二者差异不明显；当输出 20 位小数时，二者保存的近似值差异就表现出来了。

因此，输出结果不同并不是输出函数出错，而是 `float` 和 `double` 在内存中保存的近似值本身就不同。

问题 4：为什么 float 情况下输出的结果会比原来的大？这到底有没有根本性原因还是随机发生的？为什么会出现这样的情况？

`float` 输出结果比原来的十进制数大，并不是随机现象，而是二进制浮点表示和舍入规则共同作用的结果。

计算机中的 `float` 只能使用有限个二进制位保存小数。但许多十进制小数无法用有限二进制小数精确表示。例如：

$0.1(\text{十进制}) = 0.000110011001100110011\dots (\text{二进制})$

这个二进制表示是无限循环的，而 `float` 的尾数位数有限，所以只能选择一个最接近的可表示浮点数保存。真实值与保存值之间的差异称为舍入误差。

浮点数的舍入并不是随意进行的，而是由舍入模式决定的。常见默认模式是“舍入到最接近的值，平衡取偶”，即 `round to nearest, ties to even`。它会优先选择离真实值最近的可表示浮点数；如果真实值恰好位于两个可表示数的正中间，则选择末位为偶数的那个，以减少长期计算中的统计偏差。

以 `0.1`、`0.7` 和 `0.5` 为例，可以通过下面的程序观察三种情况：

```
#include "stdio.h"

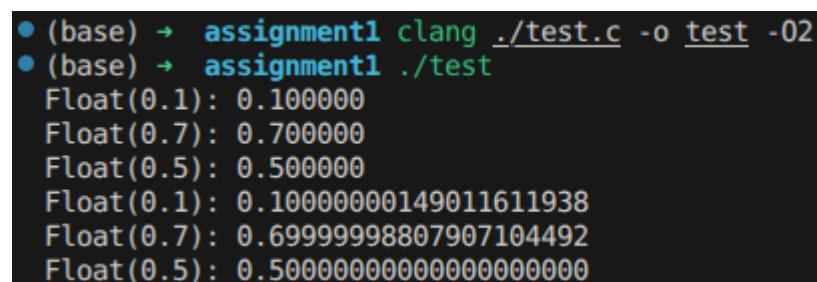
int main() {
    float a = 0.1;
    float b = 0.7;
    float c = 0.5;

    printf("Float(0.1): %f\n", a);
    printf("Float(0.7): %f\n", b);
    printf("Float(0.5): %f\n", c);

    printf("Float(0.1): %.20f\n", a);
    printf("Float(0.7): %.20f\n", b);
    printf("Float(0.5): %.20f\n", c);

    return 0;
}
```

实际运行结果如图 4 所示：



```
• (base) → assignment1 clang ./test.c -o test -O2
• (base) → assignment1 ./test
Float(0.1): 0.100000
Float(0.7): 0.700000
Float(0.5): 0.500000
Float(0.1): 0.100000000149011611938
Float(0.7): 0.69999998807907104492
Float(0.5): 0.500000000000000000000
```

对应输出为：

```
Float(0.1): 0.100000
Float(0.7): 0.700000
Float(0.5): 0.500000
Float(0.1): 0.100000000149011611938
Float(0.7): 0.69999998807907104492
Float(0.5): 0.500000000000000000000
```

由运行结果可知，默认只输出 6 位小数时，`0.1`、`0.7` 和 `0.5` 的误差不明显；当输出 20 位小数时，差异就显现出来。`0.1` 在 `float` 中实际保存的值略大于原数，`0.7` 在 `float` 中实际保存的值略小于原数，而 `0.5` 可以

被精确表示。

出现这种差异的原因是：真实的 0.1 和 0.7 都位于两个相邻的 `float` 可表示数之间。 0.1 按照舍入规则落到了右侧的可表示值，因此结果略大； 0.7 按照舍入规则落到了左侧的可表示值，因此结果略小。 0.5 等于 $1/2$ ，可以写成有限二进制小数，因此能够被精确表示。

因此，并不是所有十进制小数转换为 `float` 后都会变大。有些数会被舍入到略小的值，有些数则可以被精确表示。例如， 0.5 、 0.25 、 0.125 分别等于 $1/2$ 、 $1/4$ 、 $1/8$ ，可以用有限二进制小数精确表示。

综上，`float` 输出结果偏大或偏小都有明确原因：十进制小数无法精确转换为有限位二进制浮点数，经过舍入后可能落到比原数稍大的可表示值，也可能落到比原数稍小的可表示值。具体偏大还是偏小，取决于原数在两个相邻可表示浮点数之间的位置以及当前采用的舍入模式。

如果程序需要精确保存十进制小数，例如金额计算，就不应该直接依赖普通二进制浮点数。更合适的做法是使用十进制浮点类型、专门的 decimal 库，或者采用定点数方法，例如把金额统一转换成“分”这样的整数单位进行保存和计算。